DHBW Ravensburg

Intelligence Engineering

Solving Real World Problems At Scale

Prof. Dr.-Ing. Mark Schutera | October 18, 2026 · *drifting stream*

LECTURE _

Orientation

what this course is, what it assumes, what yor are going to know after.

You have been given a lot of raw **data**,

and you want to engineer the shit out of it.

You have been given a business problem,

and you want to choose the right metrics and objectives to solve it.

Your initial models perform well in offline experiments,

and you want to deploy them at scale.

You want to harden your deployments

to quickly detect, debug, and address issues that arise in production.

There is a lot of manual work and handling involved so far,

and you want to operation vacation your machine learning product or service.

You want to learn about a framework and foundation for building machine learning systems

that can be shared and reused across machine learning use cases and data products.

1

Overview

Machine Learning and Data Engineering

The lecture in one slide

Lecture	Contents
I	Overview of Machine Learning Systems
II	Introduction to Machine Learning Systems Design
III	Data Engineering Fundamentals
IV	Training Data
V	Feature Engineering
VI	Model Development and Offline Evaluation
VII	Model Deployment and Prediction Service
VIII	Data Distribution Shifts and Monitoring
IX	Continual Learning and Test in Production
X	Infrastructure and Tooling for MLOps
XI	The Human Side of Machine Learning

The lecture in one slide

I Overview of Machine Learning Systems

The small part and the big picture

Understanding Machine Learning Systems

II Introduction to Machine Learning Systems Design

Business and ML Objectives

Requirements for ML Systems

Iterative Process

Framing ML Problems

Mind Versus Data

III Data Engineering Fundamentals

Data Sources

Data Formats

Data Models

Data Storage Engines and Processing

Modes of Dataflow

Batch Processing Versus Stream Processing

The lecture in one slide

IV Training Data

Sampling
Labeling
Class Imbalance
Data Augmentation

V Feature Engineering

Learned Features Versus Engineered Features
Common Feature Engineering Operations
Data Leakage
Engineering Good Features

VI Model Development and Offline Evaluation

Model Development and Training
Model Offline Evaluation

The lecture in one slide

VII Model Deployment and Prediction Service

Machine Learning Deployment Myths

Batch Prediction Versus Online Prediction

Model Compression

ML on the Cloud and on the Edge

VIII Data Distribution Shifts and Monitoring

Causes of ML System Failures

Data Distribution Shifts

Monitoring and Observability

IX Continual Learning and Test in Production

Continual Learning

Test in Production

The lecture in one slide

X Infrastructure and Tooling for MLOps

Storage and Compute

Development Environment

Resource Management

ML Platform

Build Versus Buy

XI The Human Side of Machine Learning

User Experience

Team Structure

Responsible AI

Course Work

Presentation (15 min) + Q&A (open)

You get recordings of a bee hive entrance, and you are to contribute by data engineering, an annotation policy, a prediction service or specialized tooling.

Open whiteboard, three slides or one page only. Be ready when the class starts.



The bee

Course Work

What I am looking for ..

Introduction by Motivation and Literature

Method by Explanation and or Demonstration

Results show what was achieved and how well

Discussion on findings, limitations and what's next

Smart and Aesthetic Thoughts, Designs, Ideas and Questions.



The bee

Extra Mile

Special projects

If you are in for an extra mile, let me know. I have special projects that I can assign to you.

Wrapped in the same presentation and Q&A format, yes it will be about Bananas, and yes it will be more challenging, more time and effort.

Classic loose loose short term, yet **win long term**.



A large banana



By the end of this lecture,

Get yourself a presentation slot and brainstorm a project idea to work on.

LECTURE I

Overview of a Machine Learning System

This lecture in one slide

I Overview of Machine Learning Systems

The small part and the big picture

Understanding Machine Learning Systems

A **model** is only a small part from the system around it, and some might argue there are a lot of moments when not to reach for machine learning at all.

1

The small part and the big picture

Don’t confuse the model with the machine learning system.

Enabling the delivery of the logic

The **model** is the small part of a machine learning system. With foundation models and AI engineering, that part got smaller still. The essential work of a machine learning system remains and stacks around business understanding, infrastructure, design pattern and data engineering.

Before learning about solving a problem with machine learning, let's first assess when it is actually a good idea to do so.

(1) Learn

The problem you are solving, requires your system to have the capability to **learn**. Learning can look different depending on the problem and its context and boundary conditions. You already know about supervised learning, unsupervised learning and reinforcement learning paradigms.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(2) Pattern

Learning requires the existence of a **pattern**, as opposed to noise or randomness. It is not trivial nor obvious whether there is a pattern in the data, and whether it is learnable. Complex pattern is more difficult to embed or state explicitly, this is where machine learning has an edge over classical approaches.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(3) Data

Data is available, accessible or acquirable. The data is of **sufficient** quality and quantity to support learning, (and to validate for that matter). If your approach does not require data, it is because you build on embeddings or foundation models, which themselves are built on data in the first place, or you need to rely on data acquisition and curation as part of your system.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(4) Predictions

Machine learning is about making predictions, **always**. Predictions can be of different types, such as classification, regression, ranking, clustering, generation and on and on. By reframing problems in terms of predictions, we can leverage machine learning to solve them. Predictions are approximations by nature and therefore come with uncertainty, which needs to be accounted for.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(5) Unseen Data

In a closed system or domain, the data is finite and the system can be fully specified by optimization. However, often the domain is open and infinite, and the system needs to be able to **generalize** to unseen data. Of course, the unseen data is not completely unknown, it is drawn from the same distribution (i.i.d.) as the training data and is thus able to generalize to the unseen data inside a specific domain or distribution.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

Catalysts

A catalyst is a substance that increases the rate of reaction without undergoing change itself.

The task is repetitive.

In a sense that we have a lot of data to learn a pattern from.

It scales.

The need to scale a specific prediction capability, not only a few times.

The context changes.

Meaning that hard coded or other solutions will be brittle and break.

Being wrong is not too bad.

Approximations on unseen data. Failures need to be manageable.

When Not to Use Machine Learning

It is unethical.

Well, that might also be a general rule. But doing stuff at scale, without human oversight, has its implications.*

There's a simple solution.

A simple solution, heuristic, or baseline should always be the first step regardless.

It's not economically viable.

The value of the prediction, needs to be greater than the cost of the system, including development, deployment and maintenance.

* More? Enroll to my Philosophy of AI course.

Describe two use-cases and the rationale for whether or not to use machine learning.

One Machine Learning Use-Case and One which is not.

Mapping the value, 2018

The map: value in the trillions

McKinsey sized **400+** use cases across **19** industries and nine business functions: deep learning promised **$3.5 to $5.8 trillion** in value per year, the largest shares in marketing and sales plus supply chain and manufacturing.

The ground: months per model

Algorithmia surveyed **745** practitioners in October 2019: most companies needed **8 to 90 days** to deploy a single model, 18% needed longer than 90 days, and deployment time grew year over year.

The value was mapped in trillions while shipping one model took a quarter: feasibility, not value, was the bottleneck.

[2, 3]

Value over technical feasibility

SWE-bench Verified went from 60% to near 100% in a year. The frontier between hard today and works today **moves right** every quarter.

- **Deploy now** coding agents, recommender systems, fraud detection, machine translation
- **Invest** autonomous driving, medical diagnosis, agentic workflows end to end
- **Automate cheaply** speech transcription, document OCR
- **Skip** bespoke one off models

Eisenhower: value over technical feasibility

The field is moving, 2026

Adoption is near universal

88% of organizations use AI in at least one business function, 72% use gen AI. Gen AI reached 53% of the population in three years, faster than the PC or the internet.

Value capture is not

80% of users report personal productivity gains. Only **37%** of organizations can attribute any EBIT impact to AI, flat since 2025, and 6% qualify as high performers.

Adoption climbs, value capture flat

OpenAI measures the work

GDPval takes **1,320** real deliverables from **44** occupations across nine sectors, and has experts averaging 14 years in the field grade them head to head against the model.

Frontier models finish them roughly **100**× faster and **100**× cheaper. What is left is judgment, not speed.

OpenAI, GDPval, 2025 [6]. A benchmark built out of what people are actually paid to produce.

GDPval: 1320 tasks, 44 occupations

Anthropic measures the traffic

The Economic Index reads Claude's own live traffic rather than a benchmark. **49%** of occupations show Claude on at least a quarter of their tasks.

Augmentation leads automation **52% to 45%**, and the single most common task is modifying software to correct errors, 6% of all usage.



Anthropic Economic Index, 2026 [7]. Broad across occupations, thin within them, and the densest use is code.

Economic Index: where Claude is used

2

Understanding Machine Learning Systems

Research and Production are not the same.

Horizontal versus Vertical

Nothing motivates system thinking more than the need to go horizontal. Academia is vertical, with a narrow focus on a single problem, where drilling deep, fast is rewarded and focus allows to teach a wide range of concepts.

	Research	Production
Objectives	State of the art, highly engineered	Stakeholder outcome driven
Compute	Training and high throughput	Fast inference with low latency
Data	Often static benchmark oriented	Shifting domains and structures
Usability metrics	Usually not considered	UI, Fairness, Robust, Interpretability

Stakeholders and Objectives

In research the objective is one number on one benchmark. Ship the same model as a product and five people want five different things from it: **conflicting requirements** are the norm, and not all of them are must haves.



ML engineers



Platform team



Product team



Sales team



Manager

Tinder for horses

Different stakeholders have different objectives and think in different solution spaces.

Define the next step for horse tinder and line out how to reach that.

Five people, five solution spaces

Each of them is right. The **objective** is what they are measured on, the solution space is where they will look for it.

	Objective	Solution space
ML engineers	better matches	more data, a bigger model
Platform team	nothing pages at 3am	freeze the model, build the platform
Product team	the swipe feels instant	a latency budget of 100 ms
Sales team	revenue per match	rank the premium stallions first
Manager	margin	fewer models, and maybe fewer engineers

Leaderboards and Benchmarks

- **System** A leaderboard scores a **model**, and a system is not a model.
- **Metric** One metric on one frozen test set. Whether it measures what it claims is rarely argued.
- **Overfit** Every run against a benchmark stales it. GLUE was saturated within a year.
- **Chance** Scanning 10,000 seeds moves CIFAR 10 by 1.82 points, nothing else changed.

GLUE 2018. Trust me bro, nine tasks

[8-11]

Computational priorities

Research optimises **throughput**, production optimises **latency**, and one buys the other.

Processing	Latency	Throughput
One query at a time	10 ms	100 queries/s
Batches of 10	20 ms	500 queries/s
Batches of 100	50 ms	2,000 queries/s

Every row is the same model on the same hardware. Only the batch changed.

Batching waits for the queue to fill, so each single request waits longer while the machine as a whole serves far more of them.

Training leans throughput: nobody is waiting on one gradient step. Serving leans latency: somebody is.

Batch size is not the only lever. Model size, sample size and how many models sit in one request all move both numbers, and they do not move them in the same direction.

[1]

Latency is a distribution

Ten requests, latency in ms:

9.5, 9.7, 9.9, 10.0, 10.0, 10.0, 10.2, 10.4, 11.0, **90.0**

The mean is 18.1 ms. Nine of the ten requests were served in about 10 ms and not one of them took 18. The average describes a request that never happened.

Users get less patient.

2017, Akamai: a 100 ms delay can cost 7% of conversions.

2019, Booking.com: about 30% more latency cost about 0.5% in conversion, “a relevant cost for our business”.

2016, Google: over half of mobile users leave a page that takes more than three seconds.

[12–14]

Percentiles, not averages

The same ten requests. The **median** is 10.0 ms, p90 is 11.0 ms, p99 is 90.0 ms.

$$p_q \;=\; \inf\Big\{\, t \;:\; F_\ell(t) \;\geq\; \tfrac{q}{100} \,\Big\}$$

A percentile is a promise: q percent of requests finish in p_q or less. The mean hides the outlier, the percentile is the outlier.

What you actually work on is the gap between two percentiles, the slowest tenth that is not the worst hundredth:

$$S_{90,99} \;=\; \{\, r \;:\; p_{90} \leq \ell(r) < p_{99} \,\}$$

That set is big enough to hold a repeatable cause and small enough to count. $S_{99,100}$ is usually one broken machine.

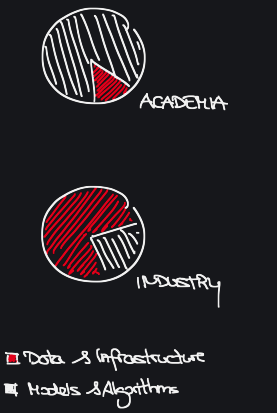
Data in research

A research dataset is a teaching artefact. Somebody already did the work.

- Clean and well formatted
- Static, so the community can benchmark against it
- Quirks of the dataset are known and documented
- Open source scripts to feed your models
- Historical, already stored somewhere

Titanic [15]: 891 rows, 12 columns, and every one of its quirks is written up in a tutorial somewhere.

Karpathy: at Tesla you lose sleep over data



[16]

Data in production

Nobody did the work. The data is a by-product of a business that was not run for your benefit.

- Messy, noisy, possibly unstructured
- **Likely biased, and you do not know how**
- Labels sparse, imbalanced, or incorrect
- Changing requirements can mean relabeling
- Privacy and regulatory concerns
- Generated by users, systems, third parties

Perpetuating Bias

Machine learning algorithms do not predict the future, they encode the past, and then they act on it.

The invisible hand of performative prediction at scale: the prediction is not a reading of the world, it is a push on it.

2014 to 2018, Amazon built a résumé ranker on ten years of its own hiring. It learned to penalise the word “women’s” and graduates of two women’s colleges. The project was scrapped.

2015, Google Photos labelled two Black users as gorillas. The fix shipped, and three years later, was still to delete the label from the vocabulary.

[17–20]

Interpretability

Research rewards performance, not interpretability. Industry asks for it anyway, from two directions:

- **Users**, business leaders and end users alike, to see why a decision was made, so they can trust it and catch what is wrong with it.
- **Developers**, to debug and improve the model at all.

Most of us use a fridge without understanding it. Fewer feel that way about the thing deciding on their mortgage. Only 19% of large companies are working on explainability.

Press 2: hand Claude recursive permission

Human on the loop

Putting a human on the loop is the standard answer to an uninterpretable model. The evidence that it works is **thin**.

Supervising reliable automation degrades exactly the capability being supervised: vigilance falls, response to the first failure is poor, and the takeover you designed for arrives too late.

Victor et al., 2018: drivers supervising automation drove into a conflict object with **eyes on the threat** and **hands on the wheel**. Looking at it was not the same as expecting it.

So a human on the loop is a design decision that has to be tested, not a control you get for free by writing it into the spec.

[21]

Why this is different from traditional software engineering

Traditional software engineering tests and versions **code**. A machine learning system has to test and version code, **data**, and everything the two produce together.

- How do you version a dataset, and diff two versions of one?
- How do you deploy into an environment that keeps moving?
- How do you monitor and debug a system with no stack trace?
- How do you avoid being harmful at scale?

Two books worth owning, and neither is about machine learning.

Software Engineering at Google [22]: engineering is programming integrated over time. Every question on the left is a question about time.

The Mythical Man-Month [23]: essential complexity against accidental. Data is essential complexity, and there is no silver bullet for it either.

A trained model is the small box in the middle.

You might argue that the engineering and development side of machine learning is more exciting.

Most jobs will be, and already are in productionizing machine learning systems.

LECTURE II

Introduction to Machine Learning Systems Design

This lecture in one slide

II Introduction to Machine Learning Systems Design

Business and ML Objectives
Requirements for ML Systems
Iterative Process
Framing ML Problems
Mind Versus Data

why this lecture matters, and what you can do after it

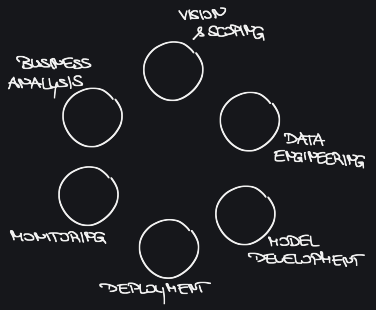
1

Business and ML Objectives

System Design

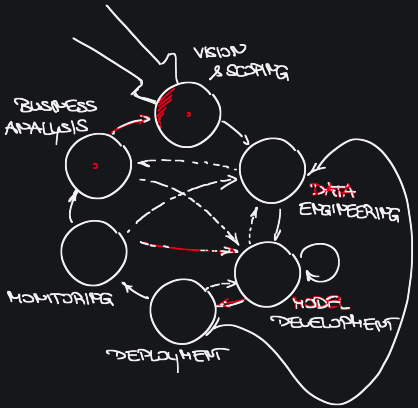
ML systems design takes a **system approach** to MLOps: the business requirements, the data stack, infrastructure, deployment, monitoring, and their stakeholders work together to satisfy the specified objectives and requirements.

If this system is built for a business, it must be driven by business objectives, which will need to be translated into ML objectives to guide the development of ML models.



[1]

It is not crazy, it is just complex



Objectives and Key Results



Objective what is to be achieved.

Key result how we get there: specific, time bound, measurable and verifiable.

One level's key result becomes the next level's objective, all the way down the org chart. The ML metric lands at the bottom.

[24]

“There is one and only one social responsibility of business … to increase its profits.”

Milton Friedman, 1970. While most companies want to convince you otherwise.

[25]

Your ML metric is a key result, very seldom the objective.

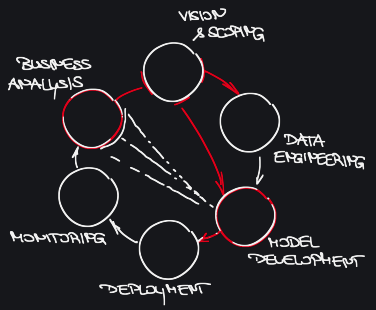
Most companies don't care about increasing a model's accuracy from 94% to 94.2% unless it moves some business metrics.

[1]

Tie ML to business

For an ML project to succeed within a business organization, it's crucial to tie the performance of an ML system to the overall business performance. At best the ML metric becomes a **proxy** for a business metric.

- **Ad click-through rate** every increase results in actual ad revenue.
- **Fraud detection** every fraudulent transaction stopped results in actual money saved.



[1]

Netflix take-rate

Netflix measures the performance of their recommender system using take-rate.

$$\text{take-rate} = \frac{\text{quality plays}}{\text{recommendations seen}}$$

The higher the take-rate, the better the recommender system.

Put in context with total streaming hours and the subscription cancellation rate: a higher take-rate also results in more streaming hours and fewer cancellations.

Over years of personalization, monthly churn fell by several percentage points, worth more than $1B per year [26].

[26, 27]

Who gains from the link

How ML metrics influence business metrics usually takes an experiment: run an **A/B test** and choose the model that leads to better business metrics, regardless of whether it has better ML metrics.

Manager

You know you pour money into a profitable initiative.

Team lead

Your ML team will not get killed with the project anytime soon.

Developer

Communicate it well and you will have a great time in performance reviews.

[1]

How important are ML systems for your company?

Rate it. Then compare it to your estimate for Netflix.

Which are the business key indicators of your company or division?

Which ML metrics can you design that are tied to them?

For all the wrong reasons

40%

of 2,830 European startups classed as AI companies showed no evidence of AI material to their value proposition.

Many companies like to say that they use ML because “being AI powered” alone already helps them attract customers, regardless of whether the AI part actually does anything useful.

Hype sells.

[28, 29]

2

Requirements for ML Systems

Forty-two

"I think the problem, to be quite honest with you, is that you've never actually known what the question is."

We can't say that we've successfully built an ML system without knowing what **requirements** the system has to satisfy.

They vary from use case to use case. Most systems should have four: reliability, scalability, maintainability, and adaptability.

Deep Thought: forty-two

[1, 30]

Reliability

Definition: Reliability

The system should continue to perform the correct function at the desired level of performance even in the face of adversity: hardware or software faults, and even human error.

Traditional software warns you: a crash, a runtime error, a 404.

ML systems can fail silently. `model.predict()` is called correctly, but the predictions are wrong, and without ground truth labels nobody knows. Translate a sentence into a language you don't know.

[1, 31]

Reliability

Definition: Reliability

The system should continue to perform the correct function at the desired level of performance even in the face of adversity: hardware or software faults, and even human error.

Fail loudly. A fault surfaces immediately and visibly, where it happens, instead of carrying on with bad state.

Degrade gracefully. Under overload, lower the quality of the response rather than drop it: a smaller index, a cheaper ranking, a simpler model.

[32, 33]

Scalability

Whichever way your system grows, there should be **reasonable ways** of dealing with that growth.

Complexity

Last year a logistic regression on 1 GB of RAM. This year a 100 million parameter network that needs 16 GB.

Traffic volume

10,000 prediction requests daily at launch. Between 1 million and 10 million once the user base grows.

Model count

One model per enterprise customer. A startup Huyen worked with had 8,000 models in production.

[1, 31]

Scaling is not only resources

Resource scaling. At peak 100 GPUs, most of the time 10. Autoscaling scales up and down with usage, and is tricky to implement.

Artifact management. One model you can monitor and retrain by hand. With one hundred, monitoring and retraining need to be automated, and every model reproducible.

Even Amazon fell victim: autoscaling failed on Prime Day 2018 and the system crashed.

One hour of downtime was estimated to cost Amazon between $72 million and $99 million.

[1, 34]

Maintainability

ML engineers, DevOps engineers, and subject matter experts, from very different backgrounds, own different parts of the process.

- Everyone works with tools they are comfortable with
- Code is documented
- Code, data, and artifacts are **versioned**
- Models are reproducible when the original authors are not around
- Problems get solved without finger pointing

"Over time, many different people will work on the system … and they should all be able to work on it productively."

[1, 31]

Adaptability

Definition: Adaptability

To adapt to shifting data distributions and business requirements, the system should have some capacity for both discovering aspects for performance improvement and allowing updates without service interruption.

ML systems are part code, part data, and data can change quickly.

Lecture VIII, Data Distribution Shifts and Monitoring. Lecture IX, Continual Learning and Test in Production.

[1]

Alignment and Controllability

Alignment

Make AI systems behave in line with human intentions and values.

Controllability

Humans can supervise the system and intervene when it deviates from the expected behavior.

Robustness, Interpretability, **Controllability**, Ethicality: the four objectives of alignment.

LLMs and agents take over decision making, or at least augment it.

[35]

In the loop, on the loop

Controllability is a design decision: **who decides**, and who acts.

Human	Decides	Acts
in the loop	human, on a proposal	system
on the loop	system, human can veto	system
out of the loop	system	system

Served to a lot of users, scale, bias and performative prediction become requirements.

Interpretability and the human machine interface decide whether the human can intervene at all. Lecture I: human on the loop.

[20, 36]

3

Iterative Process

Iterative Process

4

Framing ML Problems

Types of ML Tasks

Classification versus regression

Classification

Regression

Binary versus multiclass classification

Binary	Multiclass

Multiclass versus multilabel classification

Multiclass

Multilabel

Multiple ways to frame a problem

Objective Functions

Decoupling objectives

5

Mind Versus Data

Model Versus Data

Mind

Data

6

Summary

Summary

This is the last slide.
Look how far you've come.

References

1 Chip Huyen. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. Sebastopol, CA: O'Reilly Media, 2022.

2 Michael Chui et al. *Notes from the AI Frontier: Insights from Hundreds of Use Cases*. Discussion paper. McKinsey Global Institute, 2018.

3 Algorithmia. *2020 State of Enterprise Machine Learning*. Tech. rep. Algorithmia, Inc., 2019.

4 AI Index Steering Committee. *The AI Index 2026 Annual Report*. Tech. rep. Institute for Human-Centered AI, Stanford University, 2026.

5 McKinsey & Company. *The State of AI: Global Survey 2026*. Tech. rep. McKinsey & Company, QuantumBlack, 2026.

6 OpenAI. *GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks*. arXiv:2510.04374. 2025.

7 Anthropic. *Anthropic Economic Index Report: Economic Primitives*. Anthropic research report, January 2026. 2026.

8 Alex Wang et al. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding". In: *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*. 2018, pp. 353–355.

References

9 Inioluwa Deborah Raji et al. “AI and the Everything in the Whole Wide World Benchmark”. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. 2021.

10 Timo Freiesleben and Sebastian Zezulka. *The Benchmarking Epistemology: Construct Validity for Evaluating Machine Learning Models*. arXiv:2510.23191. 2025.

11 David Picard. *torch.manual_seed(3407) is All You Need: On the Influence of Random Seeds in Deep Learning Architectures for Computer Vision*. arXiv:2109.08203. 2021.

12 Akamai Technologies. *State of Online Retail Performance, Spring 2017*. Tech. rep. Akamai Technologies, 2017.

13 Lukas Vermeer. *Moving Fast: Latency and Conversion at Booking.com*. Booking.com engineering. 2019.

14 Google. *The Need for Mobile Speed: How Mobile Latency Impacts Publisher Revenue*. Tech. rep. Google and DoubleClick, 2016.

15 Kaggle. *Titanic: Machine Learning from Disaster*. Kaggle getting started competition. 2012.

16 Andrej Karpathy. *Building the Software 2.0 Stack*. Train AI 2018, San Francisco. 2018.

17 Jeffrey Dastin. *Amazon Scraps Secret AI Recruiting Tool That Showed Bias Against Women*. Reuters, 10 October 2018. 2018.

References

18 Tom Simonite. *When It Comes to Gorillas, Google Photos Remains Blind*. Wired, 11 January 2018. 2018.

19 Moritz Hardt. *The Invisible Hand of Prediction*. Simons Institute for the Theory of Computing, Berkeley. 2022.

20 Juan C. Perdomo et al. "Performative Prediction". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. PMLR. 2020, pp. 7599–7609.

21 Trent W. Victor et al. "Automation Expectation Mismatch: Incorrect Prediction Despite Eyes on Threat and Hands on Wheel". In: *Human Factors* 60.8 (2018), pp. 1095–1116.

22 Titus Winters, Tom Manshreck, and Hyrum Wright. *Software Engineering at Google: Lessons Learned from Programming Over Time*. Sebastopol, CA: O'Reilly Media, 2020.

23 Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Reading, MA: Addison-Wesley, 1975.

24 John Doerr. *Measure What Matters: How Google, Bono, and the Gates Foundation Rock the World with OKRs*. New York: Portfolio/Penguin, 2018.

25 Milton Friedman. *A Friedman Doctrine: The Social Responsibility of Business Is to Increase Its Profits*. The New York Times Magazine, 13 September 1970. 1970.

References

26 Carlos A. Gomez-Uribe and Neil Hunt. “The Netflix Recommender System: Algorithms, Business Value, and Innovation”. In: *ACM Transactions on Management Information Systems* 6.4 (2015), 13:1–13:19.

27 Ashok Chandrashekar et al. *Artwork Personalization at Netflix*. Netflix Technology Blog, 7 December 2017. 2017.

28 MMC Ventures. *The State of AI 2019: Divergence*. Tech. rep. MMC Ventures, in partnership with Barclays, 2019.

29 Parmy Olson. *Nearly Half of All ‘AI Startups’ Are Cashing In on Hype*. Forbes, 4 March 2019. 2019.

30 Douglas Adams. *The Hitchhiker’s Guide to the Galaxy*. London: Pan Books, 1979.

31 Martin Kleppmann. *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.

32 Jim Shore. “Fail Fast”. In: *IEEE Software* 21.5 (2004), pp. 21–25.

33 Betsy Beyer et al. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O’Reilly Media, 2016.

34 Sean Wolfe. *Amazon’s One Hour of Downtime on Prime Day May Have Cost It up to $100 Million in Lost Sales*. Business Insider, 19 July 2018. 2018.

35 Jiaming Ji, Tianyi Qiu, Boyuan Chen, et al. *AI Alignment: A Comprehensive Survey*. arXiv:2310.19852. 2023.

References

36 Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. “A Model for Types and Levels of Human Interaction with Automation”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans* 30.3 (2000), pp. 286–297.